종설

NumPy로 하는 배열 프로그래밍

https://doi.org/10.1038/s41586-020-2649-2

접수일 2020년 2월 21일

게재 승인일 2020년 6월 17일

온라인 게재일 2020년 9월 16일

오픈 액세스

업데이트 확인

Charles R. Harris[1], K. Jarrod Millman[2,3,4]✉, Stéfan J. van der Walt[2,4,5]✉, Ralf Gommers[6]✉, Pauli Virtanen[7,8], David Cournapeau[9], Eric Wieser[10], Julian Taylor[11], Sebastian Berg[4], Nathaniel J. Smith[12], Robert Kern[13], Matti Picus[4], Stephan Hoyer[14], Marten H. van Kerkwijk[15], Matthew Brett[2,16], Allan Haldane[17], Jaime Fernández del Río[18], Mark Wiebe[19,20], Pearu Peterson[6,21,22], Pierre Gérard-Marchant[23,24], Kevin Sheppard[25], Tyler Reddy[26], Warren Weckesser[4], Hameer Abbasi[6], Christoph Gohlke[27], Travis E. Oliphant[6]

배열 프로그래밍은 벡터, 행렬, 고차원 배열에 담긴 데이터에 접근하고 이를 조작하거나 연산하기 위한 강력한 구문을 제공한다. 이 구문은 간결하면서도 표현력이 높다. NumPy는 Python을 대표하는 배열 프로그래밍 라이브러리다. 물리학, 화학, 천문학, 지구과학, 생물학, 심리학, 재료과학, 공학, 금융, 경제학 등 다양한 분야의 연구 데이터 분석 파이프라인에서 필수적인 역할을 한다. 천문학을 예로 들면, 중력파를 발견하고[1] 블랙홀을 처음으로 영상화하는 데[2] 쓰인 소프트웨어 스택의 주요 구성 요소가 NumPy였다. 이 글에서는 배열의 몇 가지 기본 개념이 어떻게 과학 데이터를 정리하고 탐색하고 분석하는 단순하면서도 강력한 프로그래밍 패러다임으로 이어지는지 살펴본다. NumPy는 과학 계산용 Python 생태계의 토대다. 그만큼 널리 쓰이기 때문에 특정한 요구가 있는 사용자를 대상으로 NumPy와 비슷한 인터페이스와 배열 객체를 자체 개발한 프로젝트도 여럿 있다. NumPy는 생태계의 중심에서 이런 배열 연산 라이브러리 사이의 상호운용성을 뒷받침하는 계층 역할을 점점 더 많이 맡고 있다. 또한 응용 프로그래밍 인터페이스(API)와 함께 향후 10년간 과학과 산업 분야의 분석을 지원할 유연한 틀을 제공한다.

NumPy가 나오기 전에는 Python용 배열 패키지가 두 가지 있었다. 1990년대 중반에 개발된 Numeric은 Python에서 배열 객체와 배열을 처리하는 함수를 제공했다. Numeric은 C로 작성되었고, 선형대수의 표준 고속 구현과 연결되었다[3,4]. 초기 활용 사례로는 로런스 리버모어 국립연구소의 관성 가둠 핵융합 연구에서 C++ 응용 프로그램을 제어한 일이 있다[5]. 허블 우주 망원경에서 얻은 대형 천문 영상을 처리하기 위해 Numeric을 다시 구현한 패키지가 Numarray다. Numarray는 구조화 배열, 유연한 인덱싱, 메모리 매핑, 다양한 바이트 순서를 지원했다. 메모리를 더 효율적으로 사용하고 IEEE 754 표준에 따른 오류 처리 방식을 유연하게 선택할 수 있었으며, 타입 변환 규칙도 개선했다[6]. Numarray는 Numeric과 호환성이 높았지만 두 패키지의 차이로 사용자 커뮤니티가 갈라졌다. 그러던 2005년, Numarray의 기능에 Numeric의 작은 배열 처리 성능과 풍부한 C API를 결합해 '양쪽의 장점'을 살린 NumPy가 등장했다[7].

그로부터 15년이 지난 지금, SciPy[12], Matplotlib[13], pandas[14], scikit-learn[15], scikit-image[16]를 비롯해 과학 계산이나 수치 계산을 수행하는 거의 모든 Python 라이브러리가 NumPy를 기반으로 한다[8–11]. NumPy는 커뮤니티가 개발한 오픈 소스 라이브러리로, Python에서 쓸 수 있는 다차원 배열 객체와 그 배열을 처리하는 함수를 제공한다. NumPy 배열은 구조가 단순해 Python에서 배열 데이터를 주고받는 사실상의 표준 형식이 되었다.

NumPy는 중앙 처리 장치(CPU)를 사용해 메모리에 있는 배열을 처리한다. 최근에는 특수한 용도에 맞춘 최신 저장 장치와 하드웨어를 활용하기 위한 Python 배열 패키지가 급증했다. 이미 많은 작업이 NumPy를 기반으로 하므로, Numarray와 Numeric 시절처럼 새 라이브러리로 인해 사용자 커뮤니티가 갈라질 가능성은 훨씬 작다. 한편 커뮤니티가 새 기술과 아직 실험 단계에 있는 기술을 활용할 수 있도록 NumPy의 역할도 바뀌고 있다. 배열 프로그래밍 API를 명확히 정의하고, API 호출을 적절한 특화 배열 구현으로 디스패치하는 중앙 조정 역할을 맡고 있다.

NumPy 배열

NumPy 배열은 텐서라고도 하는 다차원 배열[17]을 효율적으로 저장하고 그 원소에 접근할 수 있는 자료 구조로, 다양한 과학 계산을 가능하게 한다. 이 구조는 메모리를 가리키는 포인터와 그곳에 저장된 데이터를 해석하는 메타데이터로 이루어진다. 대표적인 메타데이터는 '데이터 타입', '형상', '스트라이드'다(그림 1a).

[1]독립 연구자, 미국 유타주 로건. [2]캘리포니아 대학교 버클리 뇌영상센터, 미국 캘리포니아주 버클리. [3]캘리포니아 대학교 버클리 생물통계학부, 미국 캘리포니아주 버클리. [4]캘리포니아 대학교 버클리 데이터과학연구소, 미국 캘리포니아주 버클리. [5]스텔렌보스 대학교 응용수학과, 남아프리카공화국 스텔렌보스. [6]Quansight, 미국 텍사스주 오스틴. [7]이위베스퀼레 대학교 물리학과, 핀란드 이위베스퀼레. [8]이위베스퀼레 대학교 나노과학센터, 핀란드 이위베스퀼레. [9]Mercari JP, 일본 도쿄. [10]케임브리지 대학교 공학과, 영국 케임브리지. [11]독립 연구자, 독일 카를스루에. [12]독립 연구자, 미국 캘리포니아주 버클리. [13]Enthought, 미국 텍사스주 오스틴. [14]Google Research, 미국 캘리포니아주 마운틴뷰. [15]토론토 대학교 천문·천체물리학과, 캐나다 온타리오주 토론토. [16]버밍엄 대학교 심리학부, 영국 버밍엄 에지배스턴. [17]템플 대학교 물리학과, 미국 펜실베이니아주 필라델피아. [18]Google, 스위스 취리히. [19]브리티시컬럼비아 대학교 물리·천문학과, 캐나다 브리티시컬럼비아주 밴쿠버. [20]Amazon, 미국 워싱턴주 시애틀. [21]독립 연구자, 에스토니아 사우에. [22]탈린 공과대학교 사이버네틱스연구소 역학·응용수학부, 에스토니아 탈린. [23]조지아 대학교 생물·농업공학과, 미국 조지아주 애선스. [24]France-IX Services, 프랑스 파리. [25]옥스퍼드 대학교 경제학과, 영국 옥스퍼드. [26]로스앨러모스 국립연구소 CCS-7, 미국 뉴멕시코주 로스앨러모스. [27]캘리포니아 대학교 어바인 의공학과 형광동역학연구실, 미국 캘리포니아주 어바인.

✉교신저자 이메일: millman@berkeley.edu, stefanv@berkeley.edu, ralf.gommers@gmail.com

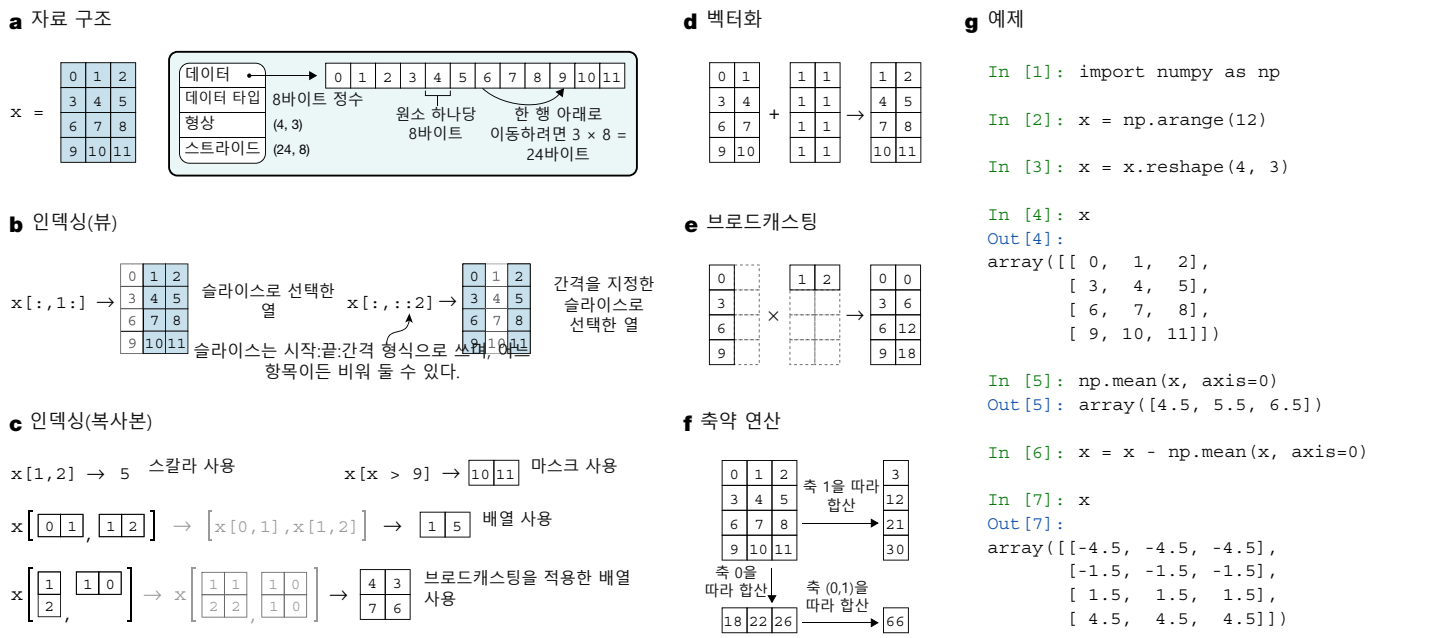


그림 1 | NumPy 배열에 담긴 기본 개념. a, NumPy 배열의 자료 구조와 메타데이터 필드. b, 슬라이스와 간격을 이용한 배열 인덱싱. 이 연산은 원본 데이터의 '뷰'를 반환한다. c, 마스크, 스칼라 좌표 또는 다른 배열로 인덱싱하면 원본 데이터의 '복사본'을 반환한다. 맨 아래 예에서는 다른 배열로 배열을 인덱싱하며, 값을 조회하기 전에 인덱스로 쓰는 배열들이 브로드캐스팅된다. d, 벡터화를 이용하면 여러 원소에 연산을 효율적으로 적용할 수 있다. e, 2차원 배열을 곱할 때의 브로드캐스팅. f, 축약 연산은 하나 이상의 축을 따라 수행한다. 예시 배열에서 한 축을 선택해 합을 구하면 벡터가 되고, 두 축을 따라 차례로 합을 구하면 스칼라가 된다. g, 이 개념들 중 일부를 보여 주는 NumPy 코드 예제.

데이터 타입은 배열에 저장하는 원소의 종류를 나타낸다. 한 배열에는 하나의 데이터 타입만 있으며, 각 원소가 메모리에서 차지하는 바이트 수도 같다. 데이터 타입의 예로는 여러 정밀도의 실수와 복소수, 문자열, 타임스탬프, Python 객체를 가리키는 포인터가 있다.

배열의 형상은 각 축을 따라 원소가 몇 개 있는지를 나타내며, 축의 개수가 곧 배열의 차원 수다. 예를 들어 수로 이루어진 벡터는 형상이 N인 1차원 배열로 저장할 수 있고, 컬러 동영상은 형상이 (T, M, N, 3)인 4차원 배열로 나타낸다.

컴퓨터 메모리에는 원소가 일렬로 저장된다. 이 메모리를 다차원 배열로 해석하려면 스트라이드가 필요하다. 스트라이드는 다음 행이나 다음 열 등으로 넘어가기 위해 메모리에서 몇 바이트를 이동해야 하는지를 나타낸다. 예를 들어 형상이 (4, 3)이고 각 원소가 메모리에서 8바이트를 차지하는 2차원 부동소수점 배열을 생각해 보자. 바로 다음 열로 가려면 메모리에서 8바이트를, 다음 행으로 가려면 3 × 8 = 24바이트를 이동해야 한다. 따라서 이 배열의 스트라이드는 (24, 8)이다. NumPy는 C의 행 우선 순서와 Fortran의 열 우선 순서 중 어느 쪽으로도 배열을 저장할 수 있다. 덕분에 두 언어로 작성한 외부 라이브러리가 메모리에 있는 NumPy 배열 데이터에 직접 접근할 수 있다.

사용자는 '인덱싱'으로 부분 배열이나 개별 원소에 접근하고, '연산자'로 계산하며(예를 들어 벡터화 연산에는 +, −, ×를, 행렬 곱셈에는 @를 사용한다), '배열을 처리하는 함수'를 호출한다. 이들이 함께 읽기 쉽고 표현력이 높은 배열 프로그래밍용 고수준 API를 구성하며, 연산을 빠르게 수행하기 위한 세부 처리는 NumPy가 맡는다.

배열을 인덱싱하면 개별 원소, 부분 배열 또는 특정 조건을 만족하는 원소를 얻을 수 있다(그림 1b). 다른 배열을 이용해 배열을 인덱싱할 수도 있다(그림 1c). 부분 배열을 가져오는 인덱싱은 가능하면 원래 배열의 '뷰'를 반환하므로 두 배열이 데이터를 공유한다. 이 방식은 메모리 사용량을 제한하면서 배열 데이터의 일부를 처리하는 데 효과적이다.

NumPy는 배열 구문과 더불어 산술, 통계, 삼각함수 등 벡터화된 계산을 배열에 적용하는 함수도 제공한다(그림 1d). 개별 원소마다 연산을 지정하는 대신 배열 전체를 대상으로 연산하는 벡터화는 배열 프로그래밍의 핵심이다. C 같은 언어에서는 수십 줄이 필요한 연산도 Python에서는 명료한 표현식 하나로 구현할 수 있는 경우가 많다. 코드는 간결해지고 사용자는 분석의 세부 내용에 집중할 수 있다. 배열 원소를 순회하는 일은 NumPy가 최적에 가까운 방식으로 처리한다. 가령 스트라이드를 고려해 컴퓨터의 고속 캐시 메모리를 최대한 활용한다.

형상이 같은 두 배열에 덧셈 같은 벡터화 연산을 적용하면 어떤 원소끼리 연산해야 하는지 분명하다. NumPy는 '브로드캐스팅'을 통해 차원이 다른 배열에도 연산을 허용하며 직관에 맞는 결과를 낸다. 배열에 스칼라 하나를 더하는 것이 간단한 예다. 이 원리는 배열의 각 열에 서로 다른 배율을 적용하거나 좌표 격자를 생성하는 등 더 복잡한 경우에도 적용된다. 브로드캐스팅에서는 두 피연산자의 형상이 일치하도록 한쪽 또는 양쪽 배열을 가상으로 복제한다. 이때 메모리의 데이터를 실제로 복사하지는 않는다(그림 1d). 인덱스 배열로 다른 배열을 인덱싱할 때도 브로드캐스팅을 적용한다(그림 1c).

sum, mean, maximum 같은 함수는 원소별로 '축약' 연산을 수행해 한 배열의 한 축, 여러 축 또는 모든 축에 걸쳐 결과를 집계한다. 예를 들어 n차원 배열에서 d개의 축을 따라 합을 구하면 결과는 n − d차원 배열이다(그림 1f).

NumPy에는 배열 생성, 형상 변경, 연결, 패딩, 데이터 검색·정렬·개수 세기, 파일 읽기·쓰기를 위한 배열 처리 함수도 있다. 의사난수 생성을 폭넓게 지원하고 다양한 확률분포를 제공한다. 또한 OpenBLAS[18],[19]나 Intel MKL처럼 사용 중인 CPU에 최적화된 백엔드 중 하나를 이용해 선형대수 연산을 가속한다(자세한 내용은 보충 자료의 연구 방법 참조).

메모리 안에서 배열을 나타내는 간단한 방식, 수학 표기에 가까운 구문, 배열을 처리하는 다양한 유틸리티 함수가 결합해 생산성과 표현력이 높은 배열 프로그래밍 언어를 이룬다.

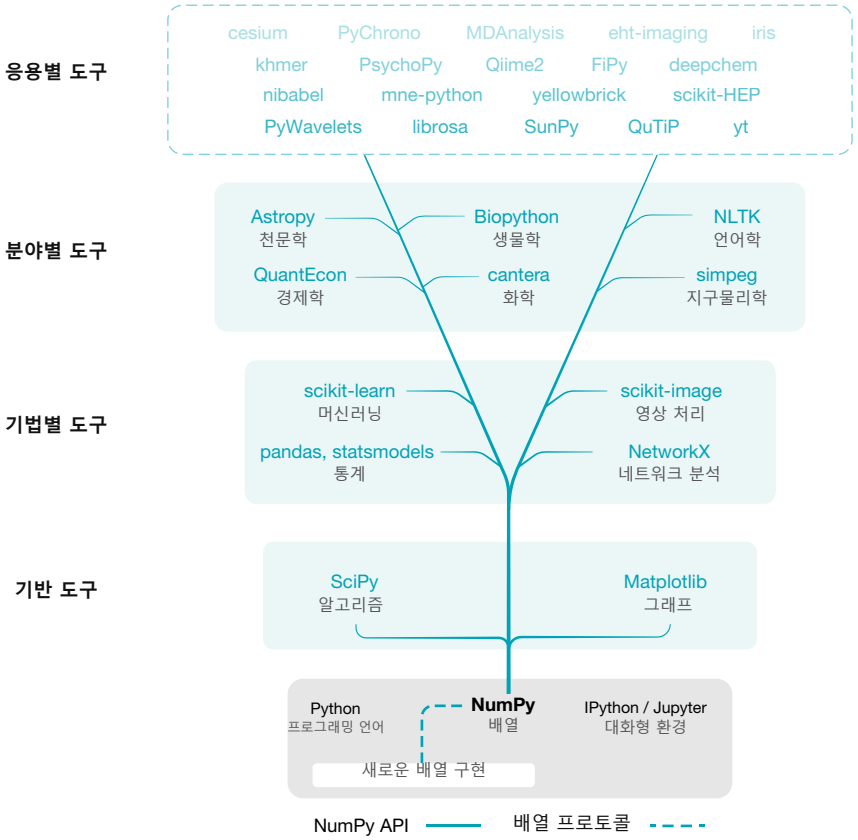


그림 2 | 과학 계산용 Python 생태계의 기반을 이루는 NumPy. NumPy API에 의존하는 주요 라이브러리와 프로젝트는 NumPy 배열 프로토콜을 지원하는 새로운 배열 구현을 사용할 수 있다(그림 3).

과학 계산용 Python 생태계

Python은 데이터 정제, 웹 자원과의 상호작용, 텍스트 파싱 등 일반적인 프로그래밍 작업에 적합한 오픈 소스 범용 인터프리터 언어다. 여기에 빠른 배열 연산과 선형대수 기능이 더해지면 과학자들은 하나의 프로그래밍 언어로 모든 작업을 수행할 수 있다. Python은 배우기도 가르치기도 쉽다는 장점이 있어 많은 대학이 프로그래밍 교육의 주 언어로 채택하고 있다.

NumPy는 Python 표준 라이브러리와 별도로 배포되지만, Python 개발자들과의 긴밀한 관계에서 이점을 얻는다. 그동안 Python에는 NumPy의 배열 표기법을 더 간결하고 읽기 쉽게 만들어 주는 기능과 특별한 구문이 추가되었다. 한편 NumPy는 표준 라이브러리에서 독립되어 있으므로 자체적인 배포 정책과 개발 방식을 정할 수 있다.

SciPy와 Matplotlib은 역사, 개발 과정, 사용 방식 모두에서 NumPy와 긴밀히 연결되어 있다. SciPy는 수학·과학·공학 계산 루틴을 비롯한 과학 계산의 기본 알고리즘을 제공한다. Matplotlib은 출판에 적합한 수준의 도표와 시각화 자료를 생성한다. NumPy, SciPy, Matplotlib은 IPython[20]이나 Jupyter[21] 같은 고급 대화형 환경과 함께 Python 배열 프로그래밍의 견고한 기반을 이룬다. 과학 계산용 Python 생태계(그림 2)는 이 토대 위에서 특정 기법에 특화된, 널리 쓰이는 라이브러리들[15,16,22]을 제공한다. 이 라이브러리들은 다시 여러 분야별 프로젝트[23–28]의 기반이 된다. 배열을 처리하는 라이브러리 생태계의 기반인 NumPy는 문서화 표준을 정하고, 배열 테스트 인프라를 제공하며, Fortran 등 여러 컴파일러를 이용한 빌드를 지원한다.

많은 연구 그룹이 특정 응용에 필요한 기능을 생태계에 더하는 크고 복잡한 과학 라이브러리를 설계해 왔다. 예를 들어 사건의 지평선 망원경(EHT) 연구 협력단이 전파 간섭계 관측 자료의 영상화·분석·시뮬레이션을 위해 개발한 eht-imaging 라이브러리[29]는 과학 계산용 Python 생태계의 여러 하위 구성 요소에 의존한다. EHT 연구 협력단은 이 라이브러리로 블랙홀을 처음으로 영상화했다. eht-imaging은 원시 데이터부터 보정, 영상 재구성에 이르는 모든 처리 단계에서 NumPy 배열을 이용해 수치 데이터를 저장하고 조작한다. SciPy는 필터링과 영상 정합 등 일반적인 영상 처리 도구를 제공한다. SciPy를 확장한 영상 처리 라이브러리 scikit-image는 경계 검출 필터와 허프 변환 등 더 높은 수준의 기능을 제공한다. 'scipy.optimize' 모듈은 수학적 최적화를 수행한다. 복잡한 네트워크를 분석하는 패키지인 NetworkX[22]는 영상 비교 결과의 일관성을 검증하는 데 쓰인다. Astropy[23,24]는 천문학 표준 파일 형식을 처리하고 시간·좌표 변환을 계산한다. Matplotlib은 데이터를 시각화하고 블랙홀의 최종 영상을 생성하는 데 쓰인다.

IPython이나 Jupyter 안에서 배열 프로그래밍의 기초 도구와 그 주위의 도구 생태계가 이루는 대화형 환경은 탐색적 데이터 분석에 특히 적합하다. 사용자는 데이터를 자유롭게 살펴보고 조작하고 시각화하면서 코드 구문을 빠르게 시험하고 다듬을 수 있다. 이렇게 다듬은 구문들을 모아 명령형 또는 함수형 프로그램을 작성하거나, 계산과 설명을 함께 담은 노트북을 만들 수 있다. 탐색 단계를 넘어서는 과학 계산은 텍스트 편집기나 Spyder 같은 통합 개발 환경(IDE)에서 수행하는 경우가 많다. 이처럼 기능이 풍부하고 생산성이 높은 환경 덕분에 과학 연구에서 Python이 널리 쓰이게 되었다.

탐색적 작업과 신속한 프로토타입 개발 여건을 뒷받침하기 위해 NumPy는 오랜 기간 검증된 소프트웨어 공학 관행을 활용해 협업을 개선하고 오류를 줄이는 문화[30]를 발전시켰다. 프로젝트를 이끄는 이들은 스스로 이 문화를 따르고 새로 참여하는 이들에게도 적극적으로 가르친다. NumPy 팀은 코드 협업을 개선하기 위해 분산 버전 관리와 코드 리뷰를 일찍 도입했고,

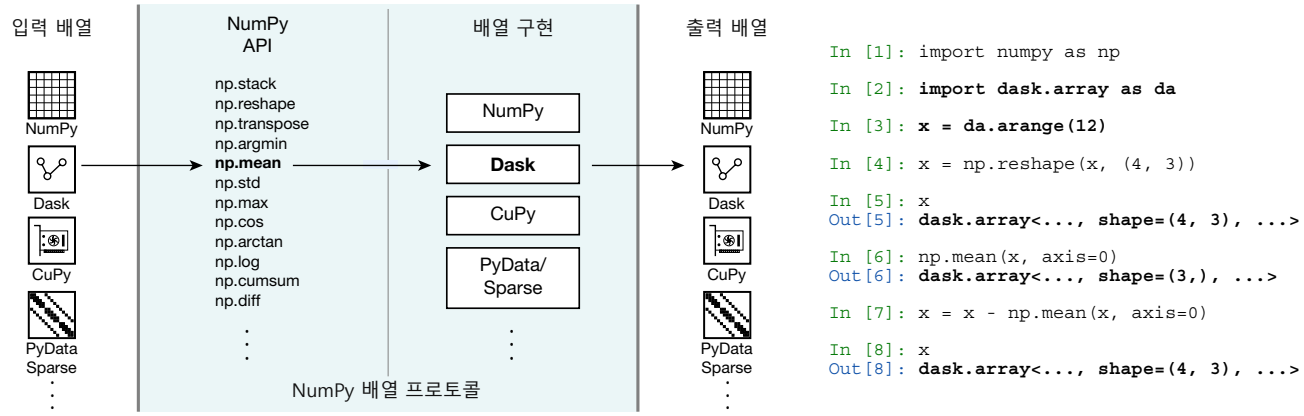


그림 3 | NumPy API와 배열 프로토콜을 통해 생태계 전체에서 새로운 배열을 사용할 수 있다. 이 예에서는 Dask 배열에 NumPy의 'mean' 함수를 호출한다. 호출이 해당 라이브러리(여기서는 Dask)의 구현으로 디스패치되므로 연산이 수행되고 새 Dask 배열이 반환된다. 이 코드를 그림 1g의 예제 코드와 비교해 보라.

NumPy에 제안된 변경 사항마다 방대한 자동 테스트를 실행하는 지속적 테스트도 도입했다. 이 프로젝트는 소스 코드와 통합된 포괄적이고 수준 높은 문서도 갖추고 있다[31–33].

신뢰할 수 있는 과학 소프트웨어를 만들기 위해 검증된 개발 관행을 따르는 문화는 NumPy를 기반으로 하는 라이브러리 생태계에도 자리 잡았다. 예를 들어 영국 왕립천문학회는 최근 Astropy에 상을 수여하면서 이렇게 밝혔다. "Astropy 프로젝트 덕분에 수백 명의 신진 과학자가 버전 관리, 단위 테스트, 코드 리뷰, 이슈 추적 절차 등 전문 개발자 수준의 소프트웨어 개발 관행을 경험했다. 이는 오늘날 연구자에게 꼭 필요한 역량이지만 대학의 물리학이나 천문학 정규 교육에서는 흔히 빠져 있다"[34]. 커뮤니티 구성원들은 강좌와 워크숍을 열어 정규 교육의 이러한 공백을 메우고 있다[35–37].

최근 데이터 과학, 머신러닝, 인공지능이 빠르게 성장하면서 과학 분야에서 Python의 활용이 한층 크게 늘었다. 이제 자연과학과 사회과학의 거의 모든 분야에서 eht-imaging 라이브러리와 같은 중요한 Python 응용 사례를 찾아볼 수 있다. 많은 분야에서 이러한 도구는 주된 소프트웨어 환경으로 자리 잡았다. 대학 강좌와 부트캠프, 여름학교에서 NumPy와 그 생태계를 흔히 가르치며, 전 세계 커뮤니티가 여는 학술대회와 워크숍에서도 이를 주요 주제로 다룬다. 이제 NumPy와 그 API는 어디서나 쓰인다.

배열 구현의 다양화와 상호운용성

NumPy는 CPU에서 연산하며 메모리에 상주하는 다차원 배열을 제공한다. 각 배열의 원소는 모두 같은 데이터 타입이며, 단일 포인터와 스트라이드로 배열을 표현한다. NumPy는 임베디드 장치에서 세계 최대 규모의 슈퍼컴퓨터에 이르기까지 다양한 기기에서 컴파일 언어에 근접하는 성능으로 실행된다. NumPy는 등장한 이래 대부분의 기간 동안 배열 계산이 필요한 거의 모든 경우를 처리할 수 있었다.

그러나 이제는 과학 데이터셋의 크기가 컴퓨터 한 대의 메모리 용량을 넘는 일이 흔하며, 여러 컴퓨터나 클라우드에 나누어 저장하기도 한다. 또한 최근 딥러닝과 인공지능 응용 프로그램의 연산을 가속할 필요가 커지면서 그래픽 처리 장치(GPU), 텐서 처리 장치(TPU), 현장 프로그래밍 가능 게이트 배열(FPGA) 등의 전용 가속 하드웨어가 등장했다. NumPy는 데이터를 메모리에 두는 모델을 사용하기 때문에 현재 이러한 저장 방식이나 전용 하드웨어를 직접 활용할 수 없다. 반면 분산 데이터 처리와 GPU, TPU, FPGA에서의 병렬 실행은 모두 배열 프로그래밍 패러다임에 잘 맞는다. 그 결과, 사용할 수 있는 최신 하드웨어 아키텍처와 그 연산 능력을 활용하는 데 필요한 도구 사이에 간극이 생겼다.

이 간극을 메우려는 커뮤니티의 노력으로 새로운 배열 구현이 다수 등장했다. 예를 들어 딥러닝 프레임워크들은 저마다 자체 배열을 만들었다. PyTorch[38], TensorFlow[39], Apache MXNet[40], JAX의 배열은 모두 CPU와 GPU에서 분산 실행될 수 있으며, 지연 평가를 활용해 성능을 추가로 최적화할 수 있다. SciPy와 PyData/Sparse는 모두 희소 배열을 제공한다. 희소 배열은 보통 0이 아닌 값이 몇 개 없으므로, 효율을 위해 그 값들만 메모리에 저장한다. NumPy 배열을 데이터 컨테이너로 활용하면서 그 기능을 확장하는 프로젝트도 있다. 이런 방식으로 Dask는 분산 배열을, xarray[41]는 레이블이 있는 배열을 구현한다. 레이블이 있는 배열에서는 각 차원을 인덱스 대신 이름으로 가리켜 의미를 분명히 할 수 있다. x[:, 1]과 x.loc[:, 'time']을 비교해 보라.

이들 라이브러리는 흔히 NumPy API를 본뜬다. 그러면 처음 사용하는 사람들의 진입 장벽이 낮아지고 커뮤니티 전체가 안정적인 배열 프로그래밍 인터페이스를 이용할 수 있기 때문이다. 이는 Numeric과 Numarray가 갈라졌을 때와 같은 혼란스러운 분열을 막아 준다. 다만 배열을 다루는 새로운 방식을 탐색하는 일에는 본래 실험적 성격이 있으며, 실제로 Theano와 Caffe처럼 유망해 보였던 라이브러리 중에는 이미 개발이 중단된 것도 있다. 또한 사용자는 새로운 기술을 시험할 때마다 import 문을 바꾸고, 새 라이브러리가 자신이 사용하는 NumPy API를 모두 구현하는지 확인해야 한다.

이상적으로는 NumPy 함수나 그 연산 규칙을 특수 목적 배열에도 그대로 적용할 수 있어야 한다. 그러면 코드를 한 번만 작성하고 필요에 따라 NumPy 배열, GPU 배열, 분산 배열 등을 바꾸어 쓸 수 있다. 이에 NumPy는 외부 배열 객체 간의 연산을 지원하기 위해 명확하게 정의된 API를 통해 중앙에서 연산을 조정하는 기능을 추가했다(그림 2).

이러한 상호운용성을 지원하기 위해 NumPy는 특수 목적 배열을 NumPy 함수에 전달할 수 있게 하는 '프로토콜', 즉 연산에 관한 약속을 제공한다(그림 3). 그러면 NumPy는 필요에 따라 해당 배열을 제공한 라이브러리로 연산을 디스패치한다. 널리 쓰이는 NumPy 함수 가운데 400개가 넘는 함수에 이 프로토콜이 적용된다. Dask, CuPy, xarray, PyData/Sparse처럼 널리 쓰이는 라이브러리들이 이 프로토콜을 구현한다. 덕분에 사용자는 예를 들어 Dask를 이용해 컴퓨터 한 대에서 하던 계산을 분산 시스템으로 확대할 수 있다. 프로토콜끼리도 잘 조합된다. 예를 들어 Dask 배열 안에 CuPy 배열을 넣어 기존 NumPy 코드를 여러 GPU로 구성된 대규모 분산 시스템에 다시 배포할 수 있다. NumPy의 고수준 API를 사용하면 코드를 조금만 수정해도 여러 시스템에 걸쳐 수백만 개의 코어를 활용하는 고도의 병렬 실행이 가능하다[42].

이러한 배열 프로토콜은 이제 NumPy의 핵심 기능이며, 그 중요성은 앞으로 더욱 커질 전망이다. 이 종설의 저자로도 다수 참여한 NumPy 개발자들은 프로토콜을 더 유용하고 쉽게 도입할 수 있도록 그 설계를 거듭 다듬고 새로운 설계를 추가하고 있다.

논의

NumPy는 배열 프로그래밍의 표현력, C의 성능, Python의 가독성과 사용 편의성, 범용성을 한데 갖춘 라이브러리다. 커뮤니티가 개발해 온 이 라이브러리는 성숙 단계에 이르렀고 테스트와 문서화도 충실하다. 과학 계산용 Python 생태계의 라이브러리는 주요 알고리즘 대부분에 대해 실행 속도가 빠른 구현을 제공한다. 극도의 최적화가 필요할 때는 Cython[43], Numba[44], Pythran[45] 같은 컴파일 언어를 사용할 수 있다. 이들 언어는 Python을 확장해 사용자가 가속 과정을 직접 다루지 않아도 병목 구간의 실행 속도를 높여 준다. NumPy의 메모리 모델은 단순하므로 보통 C나 Fortran으로 저수준 코드를 직접 최적화해 작성하고, 그 코드에서 NumPy 배열을 조작한 뒤 Python으로 돌려주기 쉽다. 또한 배열 프로토콜을 사용하면 기존 코드를 거의 바꾸지 않고도 다양한 전용 하드웨어의 가속 기능을 두루 활용할 수 있다.

처음 NumPy를 개발한 학생, 교수, 연구자들의 목적은 라이선스 서버나 소프트웨어 보호용 동글의 제약 없이 무료로 사용할 수 있는 고급 오픈 소스 배열 프로그래밍 라이브러리를 Python에 제공하는 것이었다. 많은 사람에게 도움이 될 만한 중요한 무언가를 함께 만든다는 의식이 있었다. 뜻을 같이하는 이들이 반갑게 맞아 주는 커뮤니티 안에서 이런 일에 참여할 수 있다는 점은 많은 초기 기여자에게 큰 매력으로 다가왔다.

직접 사용자이기도 했던 이 개발자들은 자신이나 동료의 문제를 해결하기 위해 코드를 처음부터 작성해야 할 때가 많았다. 그때 쓰는 언어는 Fortran[46]이나 C처럼 Python보다 먼저 나온 저수준 언어인 경우가 많았다. 따라서 대화형 고수준 배열 라이브러리의 이점은 이들에게 분명했다. 이 새로운 도구의 설계에는 Basis[47–50], Yorick[51], R[52], APL[53]처럼 과학 계산에 쓰이는 강력한 대화형 프로그래밍 언어들뿐 아니라 IDL(Interactive Data Language)이나 MATLAB 같은 상용 언어와 환경도 영향을 주었다.

Python에 배열 객체 하나를 추가하려던 시도는 활발한 도구 생태계의 토대가 되었다. 이제는 많은 과학 연구가 NumPy의 정확성, 속도, 안정성에 의존한다. 작은 커뮤니티 프로젝트로 출발한 NumPy가 과학 연구의 핵심 기반 시설이 된 것이다.

개발 문화도 성숙해졌다. 초기에는 개발 방식이 매우 비공식적이었지만, 이제 NumPy에는 개발 로드맵과 대규모 변경을 제안하고 논의하는 절차가 있다. 프로젝트는 공식적인 의사 결정 구조를 갖추고 있으며, 연구와 데이터, 과학 계산 분야의 개방적 관행을 장려하는 비영리 단체 NumFOCUS가 재정 후원 기관 역할을 한다. 지난 몇 년 동안 무어 재단과 슬론 재단의 후원으로 프로젝트는 처음으로 개발비를 확보했고, 챈 저커버그 이니셔티브의 'Essentials of Open Source Software' 프로그램에서도 지원 대상으로 선정되었다. 이 자금 덕분에 중요한 새 기능과 개선 사항을 구현하는 데 여러 달 동안 집중할 수 있었고, 지금도 그렇게 하고 있다. 그럼에도 NumPy 개발은 여전히 대학원생과 연구자들이 여가 시간을 내어 하는 기여에 크게 의존한다(자세한 내용은 보충 자료의 연구 방법 참조).

NumPy는 과학 계산용 Python 생태계를 떠받치는 기초 배열 라이브러리에서 더 나아가, 텐서 계산의 표준 API이자 Python의 여러 배열 타입과 기술을 연결하고 조정하는 구심점이 되었다. 이러한 상호운용 기능을 확장하고 개선하는 작업은 계속되고 있다.

향후 10년 동안 NumPy 개발자들은 여러 과제에 직면할 것이다. 새로운 장치들이 개발되고, 기존 전용 하드웨어는 무어의 법칙이 가져다주던 이점이 줄어드는 상황에 대응해 발전할 것이다. 데이터 과학 종사자의 수가 늘고 그 구성도 한층 다양해질 것이며, 그중 상당수가 NumPy를 사용할 것이다. 광시트 현미경과 대형 광시야 탐사 망원경(LSST)[54] 같은 장비와 관측 기기의 도입에 따라 과학 데이터 수집 규모는 계속 커질 것이다. Rust[55], Julia[56], LLVM[57]과 같은 차세대 언어, 인터프리터, 컴파일러는 새로운 개념과 자료 구조를 만들어 내고 그것들이 실제로 쓰일 수 있는지를 가름할 것이다.

이 종설에서 설명한 여러 방식에 힘입어 NumPy는 이러한 환경 변화에 대응하며 대화형 과학 계산에서 계속 주도적인 역할을 할 기반을 갖추고 있다. 다만 그러려면 정부와 학계, 산업계의 지속적인 재정 지원이 필요하다. 무엇보다 중요한 것은 앞으로 10년 동안 데이터 과학에서 요구하는 바를 충족하려면 NumPy의 발전을 이끌 새로운 세대의 대학원생과 커뮤니티 기여자가 필요하다는 점이다.

1. Abbott, B. P. 등. 두 블랙홀이 병합할 때 발생한 중력파 관측. Phys. Rev. Lett. 116, 061102 (2016).
2. Chael, A. 등. 사건의 지평선 망원경을 위한 고해상도 선편광 영상화. Astrophys. J. 286, 11 (2016).
3. Dubois, P. F., Hinsen, K. & Hugunin, J. 수치 계산을 위한 Python. Comput. Phys. 10, 262–267 (1996).
4. Ascher, D., Dubois, P. F., Hinsen, K., Hugunin, J. & Oliphant, T. E. 오픈 소스 프로젝트 Numerical Python (Lawrence Livermore National Laboratory, 2001).
5. Yang, T.-Y., Furnish, G. & Dubois, P. F. 객체 지향 과학 계산의 실행 제어. TOOLS USA 97 객체 지향 시스템 및 언어 기술 국제 학술대회 논문집 (편집 Ege, R., Singh, M. & Meyer, B.) 112–119 (IEEE, 1997).
6. Greenfield, P., Miller, J. T., Hsu, J. & White, R. L. numarray, Python을 위한 새로운 과학 계산용 배열 패키지. PyCon DC 2003. http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.112.9899 (2003).
7. Oliphant, T. E. NumPy 안내서, 제1판 (Trelgol Publishing, 2006).
8. Dubois, P. F. 기본 도구가 갖춰진 Python. Comput. Sci. Eng. 9, 7–9 (2007).
9. Oliphant, T. E. 과학 계산을 위한 Python. Comput. Sci. Eng. 9, 10–20 (2007).
10. Millman, K. J. & Aivazis, M. 과학자와 엔지니어를 위한 Python. Comput. Sci. Eng. 13, 9–12 (2011).
11. Pérez, F., Granger, B. E. & Hunter, J. D. 과학 계산 생태계로서의 Python. Comput. Sci. Eng. 13, 13–21 (2011).
과학 계산용 Python 생태계가 연구 생산성을 높이는 이유를 설명한다.
12. Virtanen, P. 등. SciPy 1.0, Python 과학 계산의 기본 알고리즘. Nat. Methods 17, 261–272 (2020), 정정 17, 352 (2020).
SciPy 라이브러리를 소개하고 NumPy와 SciPy의 역사를 더 자세히 다룬다.
13. Hunter, J. D. Matplotlib, 2차원 그래픽 환경. Comput. Sci. Eng. 9, 90–95 (2007).
14. McKinney, W. Python 통계 계산을 위한 자료 구조. 제9회 과학 분야 Python 학술대회 논문집 (편집 van der Walt, S. & Millman, K. J.) 56–61 (2010).
15. Pedregosa, F. 등. Scikit-learn을 이용한 Python 머신러닝. J. Mach. Learn. Res. 12, 2825–2830 (2011).
16. van der Walt, S. 등. scikit-image를 이용한 Python 영상 처리. PeerJ 2, e453 (2014).
17. van der Walt, S., Colbert, S. C. & Varoquaux, G. 효율적인 수치 계산을 위한 자료 구조인 NumPy 배열. Comput. Sci. Eng. 13, 22–30 (2011).
NumPy 배열의 자료 구조가 어떻게 효율적인 계산을 가능하게 하는지 설명한다.
18. Wang, Q., Zhang, X., Zhang, Y. & Yi, Q. AUGEM, x86 CPU용 고성능 밀집 선형대수 커널의 자동 생성. SC'13 고성능 컴퓨팅·네트워킹·스토리지·분석 국제 학술대회 논문집 25 (IEEE, 2013).
19. Xianyi, Z., Qian, W. & Yunquan, Z. Loongson 3A 프로세서에서 모델 기반으로 레벨 3 BLAS 성능 최적화하기. 2012년 제18회 IEEE 병렬 및 분산 시스템 국제 학술대회 684–691 (IEEE, 2012).
20. Pérez, F. & Granger, B. E. IPython, 대화형 과학 계산 시스템. Comput. Sci. Eng. 9, 21–29 (2007).
21. Kluyver, T. 등. 재현 가능한 계산 워크플로를 위한 출판 형식, 주피터 노트북(Jupyter Notebooks). 『학술 출판의 위상과 권력 — 참여자, 행위자, 의제』 (편집 Loizides, F. & Schmidt, B.) 87–90 (IOS Press, 2016).
22. Hagberg, A. A., Schult, D. A. & Swart, P. J. NetworkX를 이용한 네트워크 구조·동역학·기능 탐구. 제7회 과학 분야 Python 학술대회 논문집 (편집 Varoquaux, G., Vaught, T. & Millman, K. J.) 11–15 (2008).
23. Astropy Collaboration 등. Astropy, 커뮤니티가 개발한 천문학용 Python 패키지. Astron. Astrophys. 558, A33 (2013).
24. Price-Whelan, A. M. 등. Astropy 프로젝트의 오픈 사이언스 프로젝트 구축 과정과 v2.0 핵심 패키지의 현황. Astron. J. 156, 123 (2018).
25. Cock, P. J. 등. Biopython, 계산분자생물학과 생물정보학을 위해 자유롭게 쓸 수 있는 Python 도구. Bioinformatics 25, 1422–1423 (2009).
26. Millman, K. J. & Brett, M. Python을 이용한 기능적 자기공명영상 분석. Comput. Sci. Eng. 9, 52–55 (2007).
27. The SunPy Community 등. SunPy, 태양물리학을 위한 Python. Comput. Sci. Discov. 8, 014009 (2015).
28. Hamman, J., Rocklin, M. & Abernathy, R. Pangeo, 지구 시스템 과학의 대규모 연산을 지원하는 빅데이터 생태계. 유럽지구과학연맹(EGU) 총회 초록집 12146 (2018).
29. Chael, A. A. 등. ehtim, 전파 간섭계용 영상화·분석·시뮬레이션 소프트웨어. Astrophysics Source Code Library https://ascl.net/1904.004 (2019).
30. Millman, K. J. & Pérez, F. 오픈 소스에 기반한 과학 연구 관행의 발전. 『재현 가능한 연구의 실천』 (편집 Stodden, V., Leisch, F. & Peng, R. D.) 149–183 (CRC Press, 2014).
NumPy와 SciPy 커뮤니티가 채택한 소프트웨어 공학 관행을 소개하고, 이러한 관행이 연구를 어떻게 개선하는지 설명한다.
31. van der Walt, S. SciPy 문서화 프로젝트의 기술 개요. 제7회 과학 분야 Python 학술대회(SciPy 2008) 논문집 (편집 Varoquaux, G., Vaught, T. & Millman, K. J.) 27–28 (2008).
32. Harrington, J. SciPy 문서화 프로젝트. 제7회 과학 분야 Python 학술대회(SciPy 2008) 논문집 (편집 Varoquaux, G., Vaught, T. & Millman, K. J.) 33–35 (2008).
33. Harrington, J. & Goldsmith, D. 2009년 NumPy와 SciPy 문서화 경과 보고. 제8회 과학 분야 Python 학술대회(SciPy 2009) 논문집 (편집 Varoquaux, G., van der Walt, S. & Millman, K. J.) 84–87 (2009).
34. 왕립천문학회. 2020년 왕립천문학회(RAS) 'A'상 위원회 보고서. Astropy 프로젝트, 2020년 단체업적상(A). https://ras.ac.uk/sites/default/files/2020-01/Group%20Award%20-%20Astropy.pdf (2020).
35. Wilson, G. 소프트웨어 카펜트리, 과학자의 생산성을 높여 더 나은 코드를 작성하도록 돕기. Comput. Sci. Eng. 8, 66–69 (2006).

36. Hannay, J. E. 등. 과학자들은 과학 소프트웨어를 어떻게 개발하고 사용하는가? 2009년 ICSE 계산과학·공학을 위한 소프트웨어 공학 워크숍 논문집 1–8 (IEEE, 2009).

37. Millman, K. J., Brett, M., Barnowski, R. & Poline, J.-B. 신경영상 분야의 계산 재현성 교육. Front. Neurosci. 12, 727 (2018).

38. Paszke, A. 등. PyTorch, 명령형 고성능 딥러닝 라이브러리. 『신경정보처리시스템의 발전』 제32권 (편집 Wallach, H. 등) 8024–8035 (Neural Information Processing Systems, 2019).

39. Abadi, M. 등. TensorFlow, 대규모 머신러닝 시스템. OSDI'16 제12회 USENIX 운영체제 설계 및 구현 학술대회 논문집 (좌장 Keeton, K. & Roscoe, T.) 265–283 (USENIX Association, 2016).

40. Chen, T. 등. MXNet, 이기종 분산 시스템을 위한 유연하고 효율적인 머신러닝 라이브러리. 사전 공개 논문. http://www.arxiv.org/abs/1512.01274 (2015).

41. Hoyer, S. & Hamman, J. xarray, Python에서 레이블을 붙인 N차원 배열과 데이터셋. J. Open Res. Softw. 5, 10 (2017).

42. Entschev, P. Dask, CuPy, RAPIDS를 이용한 분산 다중 GPU 컴퓨팅. EuroPython 2019. https://ep2019.europython.eu/media/conference/slides/fX8dJsD-distributed-multi-gpu-computing-with-dask-cupy-and-rapids.pdf (2019).

43. Behnel, S. 등. Cython, 두 언어의 장점을 함께 누리다. Comput. Sci. Eng. 13, 31–39 (2011).

44. Lam, S. K., Pitrou, A. & Seibert, S. Numba, LLVM 기반 Python JIT 컴파일러. 제2회 고성능 컴퓨팅용 LLVM 컴파일러 인프라 워크숍(LLVM '15) 논문집 7:1–7:6 (ACM, 2015).

45. Guelton, S. 등. Pythran을 이용한 과학 계산용 Python 프로그램의 정적 최적화. Comput. Sci. Discov. 8, 014001 (2015).

46. Dongarra, J., Golub, G. H., Grosse, E., Moler, C. & Moore, K. Netlib과 NA-Net을 통한 과학 계산 커뮤니티 구축. IEEE Ann. Hist. Comput. 30, 30–41 (2008).

47. Barrett, K. A., Chiu, Y. H., Painter, J. F., Motteler, Z. C. & Dubois, P. F. Basis 시스템 제I부. Basis 프로그램 실행을 위한 초보자용 안내서. UCRL-MA-118543, 제1권 (Lawrence Livermore National Laboratory 1995).

48. Dubois, P. F. & Motteler, Z. Basis 시스템 제II부. Basis 언어 참조 설명서. UCRL-MA-118543, 제2권 (Lawrence Livermore National Laboratory, 1995).

49. Chiu, Y. H. & Dubois, P. F. Basis 시스템 제III부. EZN 사용자 설명서. UCRL-MA-118543, 제3권 (Lawrence Livermore National Laboratory, 1995).

50. Chiu, Y. H. & Dubois, P. F. Basis 시스템 제IV부. EZD 사용자 설명서. UCRL-MA-118543, 제4권 (Lawrence Livermore National Laboratory, 1995).

51. Munro, D. H. & Dubois, P. F. 인터프리터 언어 Yorick의 활용. Comput. Phys. 9, 609–615 (1995).

52. Ihaka, R. & Gentleman, R. 데이터 분석과 그래픽을 위한 언어 R. J. Comput. Graph. Stat. 5, 299–314 (1996).

53. Iverson, K. E. 프로그래밍 언어. 1962년 춘계 합동 컴퓨터 학술대회 논문집 345–351 (1962).

54. Jenness, T. 등. LSST 데이터 관리 소프트웨어의 개발 관행과 도구. SPIE 논문집 제10707권, 『천문학용 소프트웨어와 사이버 인프라 V』 1070709 (SPIE and International Society for Optics and Photonics, 2018).

55. Matsakis, N. D. & Klock, F. S. Rust 언어. Ada Letters 34, 103–104 (2014).

56. Bezanson, J., Edelman, A., Karpinski, S. & Shah, V. B. Julia, 수치 계산에 대한 새로운 접근법. SIAM Rev. 59, 65–98 (2017).

57. Lattner, C. & Adve, V. LLVM, 프로그램의 수명 주기 전반에 걸쳐 분석과 변환을 수행하는 컴파일 프레임워크. 2004년 코드 생성 및 최적화 국제 심포지엄(CGO'04) 논문집 75–88 (IEEE, 2004).

감사의 글. 원고에 넣을 문구를 제안하고 유익한 의견을 준 R. Barnowski, P. Dubois, M. Eickenberg, P. Greenfield에게 감사드린다. K.J.M.과 S.J.v.d.W.는 고든·베티 무어 재단과 앨프리드 P. 슬론 재단이 캘리포니아 대학교 버클리에 지급한 연구비(각각 GBMF3834, 2013-10-27)로 일부 지원을 받았다. S.J.v.d.W., S.B., M.P., W.W.는 두 재단이 같은 대학교에 지급한 연구비(각각 GBMF5447, G-2017-9960)로 일부 지원을 받았다.

저자 기여. K.J.M.과 S.J.v.d.W.가 다른 이들의 의견을 받아 원고를 작성했다. S.B., R.G., K.S., W.W., M.B., T.R.이 일부 내용을 집필했다. 모든 저자가 코드 작성, 문서화, 전문 지식 제공 중 한 가지 이상의 방식으로 NumPy 프로젝트에 실질적으로 기여했다. 모든 저자가 원고를 검토했다.

이해관계. 저자들은 상충하는 이해관계가 없다고 밝힌다.

추가 정보

이 논문의 보충 자료는 https://doi.org/10.1038/s41586-020-2649-2 에서 확인할 수 있다.
논문에 관한 문의와 자료 요청은 K.J.M., S.J.v.W. 또는 R.G.에게 보내면 된다.
동료 심사 정보. Nature는 이 논문의 동료 심사에 기여한 Edouard Duchesnay, Alan Edelman 및 익명의 심사자에게 감사드린다.
별쇄본과 이용 허가에 관한 정보는 http://www.nature.com/reprints 에서 확인할 수 있다.
출판사 주. Springer Nature는 게재된 지도와 표기된 소속 기관에 관한 관할권 주장에 대해 중립을 지킨다.

CC BY(저작자표시)

오픈 액세스. 이 논문에는 크리에이티브 커먼즈 저작자표시 4.0 국제 라이선스가 적용된다. 원저자와 출처를 적절히 밝히고, 크리에이티브 커먼즈 라이선스 링크를 제공하며, 변경한 경우 이를 표시하면 어떤 매체나 형식으로든 이용, 공유, 개작, 배포, 복제할 수 있다. 이 논문에 실린 이미지와 기타 제3자 자료에도 해당 자료의 출처 표기에 달리 명시되어 있지 않으면 이 논문의 크리에이티브 커먼즈 라이선스가 적용된다. 이 라이선스가 적용되지 않는 자료를 이용하려는 경우, 그 이용이 법령상 허용되지 않거나 허용된 범위를 넘으면 저작권자로부터 직접 허가를 받아야 한다. 라이선스 전문은 http://creativecommons.org/licenses/by/4.0/ 에서 확인할 수 있다.

© 저자 2020